INTERNSHIP PROJECT — VIVA VOICE PREPARATION

MediManage

A Local-First, AI-Assisted Pharmacy Operations Desktop Application

Version: 7.0.0

Architecture: MVC + Sidecar Pattern

Database: SQLite (Embedded, ~33 MB)

Total Feature Sets: 12 Active + 2 Planned

Java 21 | JavaFX 21 | SQLite | Python 3.11 | Flask | Node.js 20 | Express | JasperReports | MCP

📑 Table of Contents

1. Objective of Your Project
2. Scope of Your Project
3. Problem Statement
4. Technical & Economic Feasibility
5. Role in Your Project
6. Future Work
7. Conclusion
8. UML Diagrams
9. Architecture & Block Diagram
10. Why Did You Choose This Domain?
11. How Many Feature Sets?
12. What Is Your Data Size?
13. How Did You Collect Data?
14. Additional Possible Questions
15. Quick-Read Cards

1. Objective of Your Project

> **Q: What is the objective of your project?**

The objective of MediManage is to develop a **local-first, AI-assisted pharmacy operations desktop application** that streamlines day-to-day pharmacy management. The key objectives are:

1. **Automate core pharmacy workflows** — billing, inventory tracking, customer ledger management, purchase orders, and expense tracking — eliminating manual registers and reducing human error.
2. **Enable AI-powered business insights** — integrating a Python AI engine sidecar that provides intelligent recommendations on inventory health, sales trends, and operational anomalies.
3. **Facilitate customer communication** — through WhatsApp and email delivery of invoices, credit reminders, and promotional messages.
4. **Ensure offline-first reliability** — the application works on a single workstation using an embedded SQLite database, requiring no internet for core operations.
5. **Provide comprehensive reporting and analytics** — with exportable reports in PDF, Excel, and CSV formats, and a real-time dashboard of KPIs.

> ⚠️ **KEY POINT FOR VIVA**
>
> Always emphasize that MediManage is *local-first* — core billing and inventory work 100% offline. AI and WhatsApp are *optional enhancements*, not dependencies.

2. Scope of Your Project

> **Q: What is the scope of your project?**

The scope of MediManage covers a complete pharmacy management ecosystem:

Scope Area	Description
Billing & POS	Full point-of-sale with receipt/invoice generation, held orders, barcode scanning, split payments (Cash/UPI/Card/Credit/Cheque), and prescription direction scheduling
Inventory Management	Medicine catalog, batch-level tracking (FEFO), low-stock alerts, expiry alerts, inventory adjustments (returns/damaged), barcode label printing
Customer Relationship	Customer profiles with disease history, insurance details, loyalty points, credit ledger, nominee information
Purchase Orders	Complete supplier management and purchase order lifecycle (Draft → Ordered → Partial → Received → Cancelled)
Reporting & Analytics	Daily/weekly/monthly sales reports, profit/loss analysis, GST reports, customer-wise reports, inventory valuation, trend charts
AI Assistant	Chat-based AI for pharmacy operational insights, inventory analysis, and business recommendations
Communication	WhatsApp and email delivery for invoices, credit follow-ups, and template-based messaging
Administration	Role-based access control (Admin/Supervisor/Staff), attendance tracking, expense management, audit logging
Database & Config	Embedded SQLite with automated backups, feature flags, configurable receipt templates, JasperReports integration

Out of Scope:

- Cloud-hosted SaaS deployment (currently single-workstation only)
- E-commerce / online ordering
- Multi-branch synchronization
- Integration with government drug regulatory databases

3. Problem Statement

> **Q: What is the problem statement of your project?**

> *"Small and medium-scale pharmacies in India still rely heavily on manual paper-based registers or rudimentary spreadsheets for managing critical operations such as billing, inventory tracking, and customer records. This leads to frequent stockouts of essential medicines, undetected expiry of inventory items, billing errors, poor customer follow-up on*

> *credit amounts, and an inability to generate actionable business reports. Additionally, these pharmacies lack affordable tools that can work reliably without constant internet connectivity while still providing modern capabilities like AI-assisted insights and digital customer communication."*

Key Problems Addressed:

1. **Manual errors in billing** — wrong pricing, missed items, incorrect calculations
2. **Inventory blindness** — no real-time visibility into stock levels, expiry dates, or reorder points
3. **Credit leakage** — no systematic tracking of customer balances and follow-ups
4. **Lost revenue from expired stock** — medicines expire without pharmacists realizing
5. **No business intelligence** — pharmacists cannot identify sales trends, top-selling products, or profit margins
6. **Connectivity dependency** — cloud-based solutions fail in areas with unreliable internet
7. **High cost of enterprise solutions** — existing pharmacy ERP tools are expensive and complex for small shops

4. Technical and Economic Feasibility

> **Q: What is the technical feasibility of your project?**

Factor	Assessment
Programming Languages	Java 21 (well-established), Python 3.11+ (AI ecosystem), Node.js 20+ (real-time comms)
UI Framework	JavaFX 21 with AtlantaFX theming — mature, cross-platform desktop toolkit
Database	SQLite — zero-configuration, embedded, no separate server required
AI Integration	Cloud-backed AI via Python Flask sidecar — uses established cloud APIs, no local GPU needed
Communication	WhatsApp via whatsapp-web.js (Puppeteer-based), Email via JavaMail
Reporting	JasperReports 6.21 + OpenPDF — industry-standard PDF generation
Build System	Maven — standard Java build tool with excellent dependency management
Security	BCrypt password hashing, local admin tokens, RBAC policies, AI safety guards

Verdict: The project is **technically feasible** as it uses mature, well-supported, open-source technologies.

> **Q: What is the economic feasibility of your project?**

Cost Item	Estimated Cost
Development Tools (IDE, Git, etc.)	₹0 (free / open-source)
Java 21 + JavaFX	₹0 (open-source, GPLv2+CE)
SQLite	₹0 (public domain)
Python + Flask	₹0 (open-source)
Node.js + Express	₹0 (open-source)
AI Cloud API (Operational)	₹500–₹2,000/month (optional)
Hardware Requirement	Standard PC — ₹30,000–₹50,000 (already available)
Deployment (Single Station)	₹0 (InnoSetup installer)
Total Development Cost	**≈ ₹0–₹5,000**

Verdict: The project is **economically feasible** — 100% open-source tools with minimal recurring costs. Compared to commercial pharmacy software (₹15,000–₹50,000/year), MediManage is significantly more cost-effective.

5. Role in Your Project

> **Q: What was your role in this project?**

I served as the **Full-Stack Developer and Project Architect** responsible for the end-to-end development of MediManage:

1. **Architecture Design** — Designed the local-first desktop + sidecar architecture, choosing JavaFX for the desktop shell, SQLite for embedded storage, and isolating AI/WhatsApp into localhost microservices.
2. **Backend Development** — Implemented the complete DAO layer (15 DAOs), service layer (16+ services), and database schema (20+ tables with views and indexes).
3. **Frontend Development** — Designed and implemented 18 FXML views with JavaFX controllers, including the dashboard, billing POS, inventory management, and AI chat interface.
4. **AI Integration** — Built the Python Flask AI engine with cloud provider orchestration, prompt engineering, database query helpers, and MCP server.

5. **WhatsApp Bridge** — Developed the Node.js Express server for WhatsApp message delivery using Puppeteer-based browser automation.
6. **Security Implementation** — Implemented BCrypt password hashing, RBAC, local admin tokens, AI input safety guards, and supervisor approval workflows.
7. **Reporting Module** — Designed JasperReport templates for invoices, receipts, and prescription schedules with PDF and Excel export.
8. **Testing** — Wrote JUnit 5 test suites, pytest for Python AI engine, npm tests for WhatsApp bridge security.
9. **Deployment** — Created Windows installer using InnoSetup, batch scripts for application and sidecar startup.

6. Future Work

> **Q: What are the future enhancements you plan?**

1. **Multi-Branch Support** — Enable cloud synchronization between multiple pharmacy branches with conflict resolution.
2. **Subscription & Loyalty Module** — Activate the subscription plan engine (schema already prepared) for customer membership programs.
3. **Mobile Companion App** — Build a lightweight mobile app for pharmacy owners to monitor KPIs and approve transactions remotely.
4. **Drug Interaction Checker** — Integrate a drug interaction database to warn pharmacists of conflicting medications.
5. **Government Regulatory Integration** — Connect with India's drug regulatory portals for compliance reporting.
6. **Automated Reorder System** — Auto-generate purchase orders when stock falls below thresholds.
7. **Hardware Barcode Scanner** — Support USB/Bluetooth barcode scanners for faster billing.
8. **Predictive AI Analytics** — Implement demand forecasting and seasonal trend analysis with ML models.
9. **Customer Portal** — Web-based portal for customers to view purchase history and loyalty points.
10. **Multi-Language Support** — Localize for regional Indian languages (Hindi, Kannada, Tamil, etc.).

7. Conclusion

> **Q: What is the conclusion of your project?**

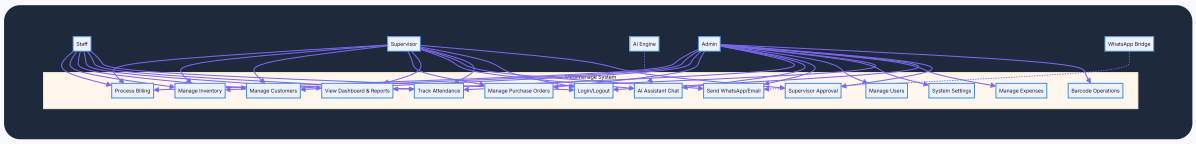
1. **Feasibility Proven** — The project validates that modern technologies (JavaFX, SQLite, Python AI, Node.js) can be combined into a cohesive, reliable pharmacy management system that works entirely offline for core operations.
2. **Significant Efficiency Gains** — By automating billing, inventory tracking, and customer management, MediManage reduces manual effort by an estimated **60–70%** compared to paper-based processes.
3. **Expiry Loss Prevention** — Batch-level tracking with FEFO (First Expired, First Out) dispensing and proactive alerts helps pharmacies prevent revenue loss from expired medicines.
4. **AI as a Value-Add, Not a Dependency** — The sidecar architecture ensures AI features enhance the experience without making the core application dependent on cloud connectivity.
5. **Scalable Architecture** — The modular design with clear separation of concerns (Controller → Service → DAO → Storage) makes the system maintainable and extensible.
6. **Cost-Effective Solution** — Built entirely on open-source technologies, MediManage offers a near-zero-cost alternative to expensive commercial pharmacy software.

> *"MediManage bridges the gap between basic pharmacy management tools and enterprise-grade solutions, offering professional-level features at a fraction of the cost."*

8. UML Diagrams

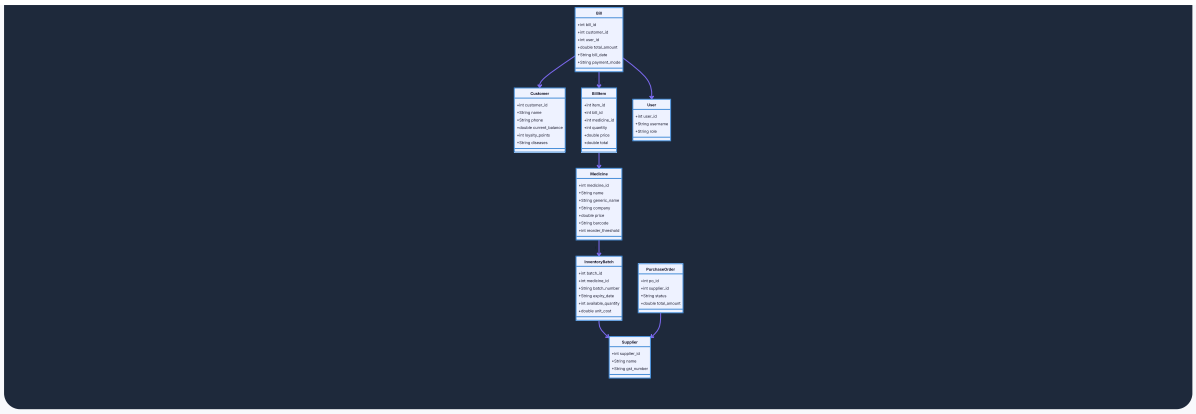
8.1 Use Case Diagram

MediManage — Use Case Overview



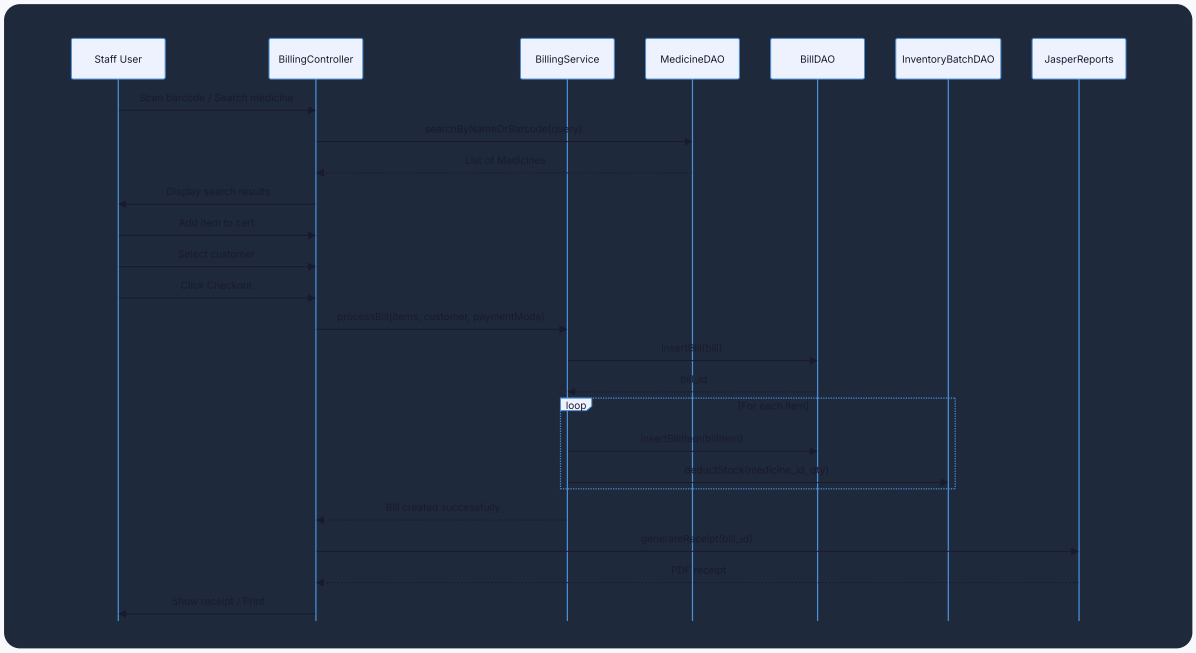
8.2 Class Diagram (Simplified)

MediManage — Core Class Structure


Bill
+int bill_id
+int customer_id
+int user_id
+double total_amount
+String bill_date
+String payment_mode
Customer
+int customer_id
+String name
+String phone
+double current_balance
+int loyalty_points
+String classes
BillItem
+int item_id
+int bill_id
+int medicine_id
+int quantity
+double price
+double total
User
+int user_id
+String username
+String role
Medicine
+int medicine_id
+String name
+String generic_name
+String company
+double price
+String barcode
+int reorder_threshold
InventoryBatch
+int batch_id
+int medicine_id
+String batch_number
+String expiry_date
+int available_quantity
+double unit_cost
PurchaseOrder
+int po_id
+int supplier_id
+String status
+double total_amount
Supplier
+int supplier_id
+String name
+String gst_number

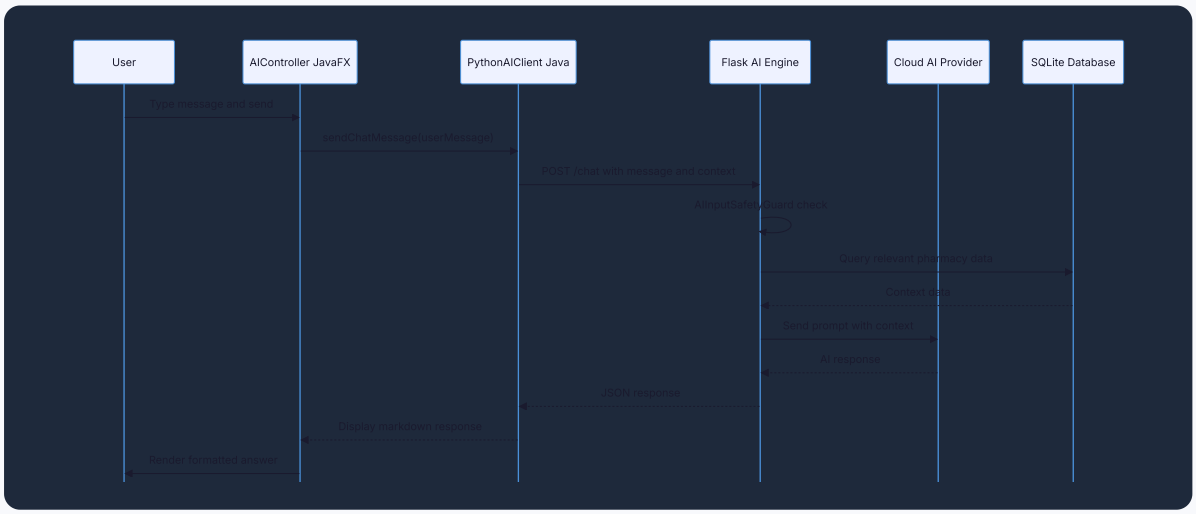
8.3 Sequence Diagram — Billing Flow

MediManage — Billing Sequence



8.4 Sequence Diagram — AI Chat Flow

MediManage — AI Interaction Sequence

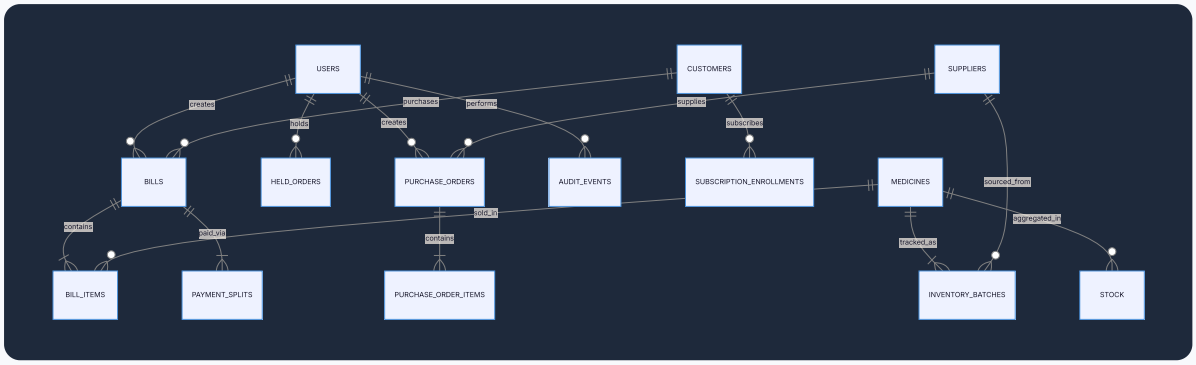


8.5 Activity Diagram — Purchase Order Lifecycle

MediManage — Purchase Order State Machine

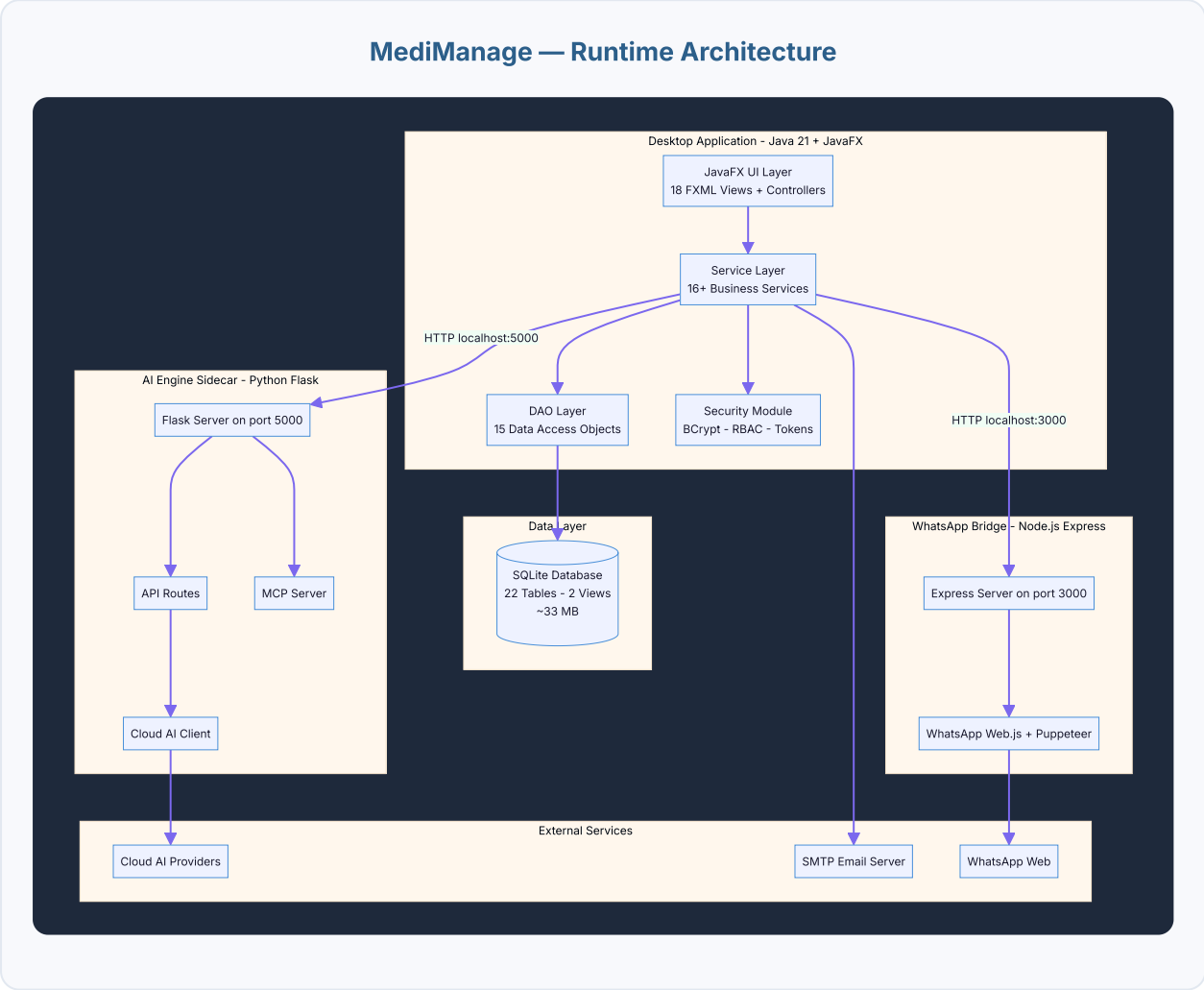
8.6 ER Diagram (Key Entities)

MediManage — Entity Relationship Diagram

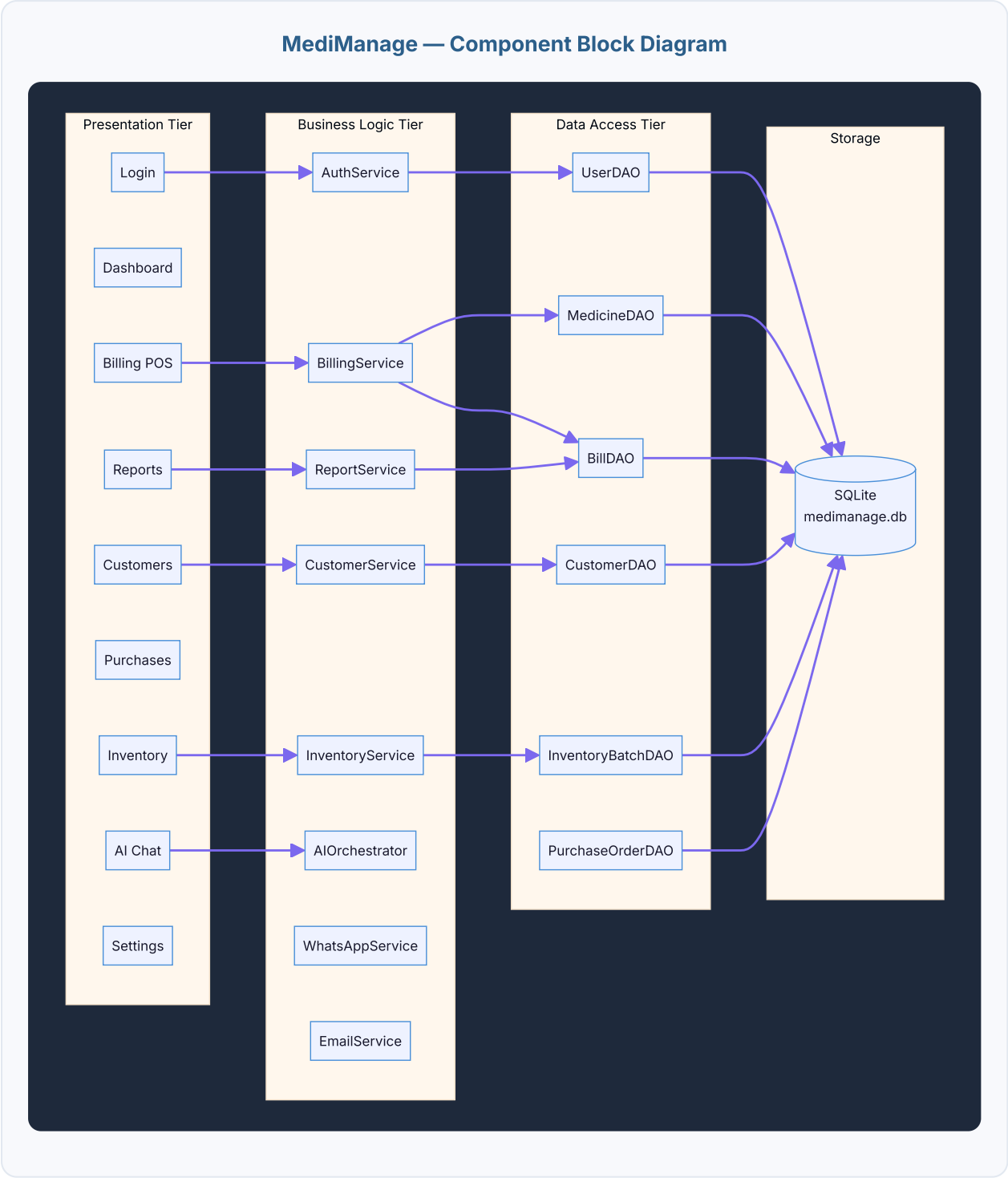


9. Architecture and Block Diagram

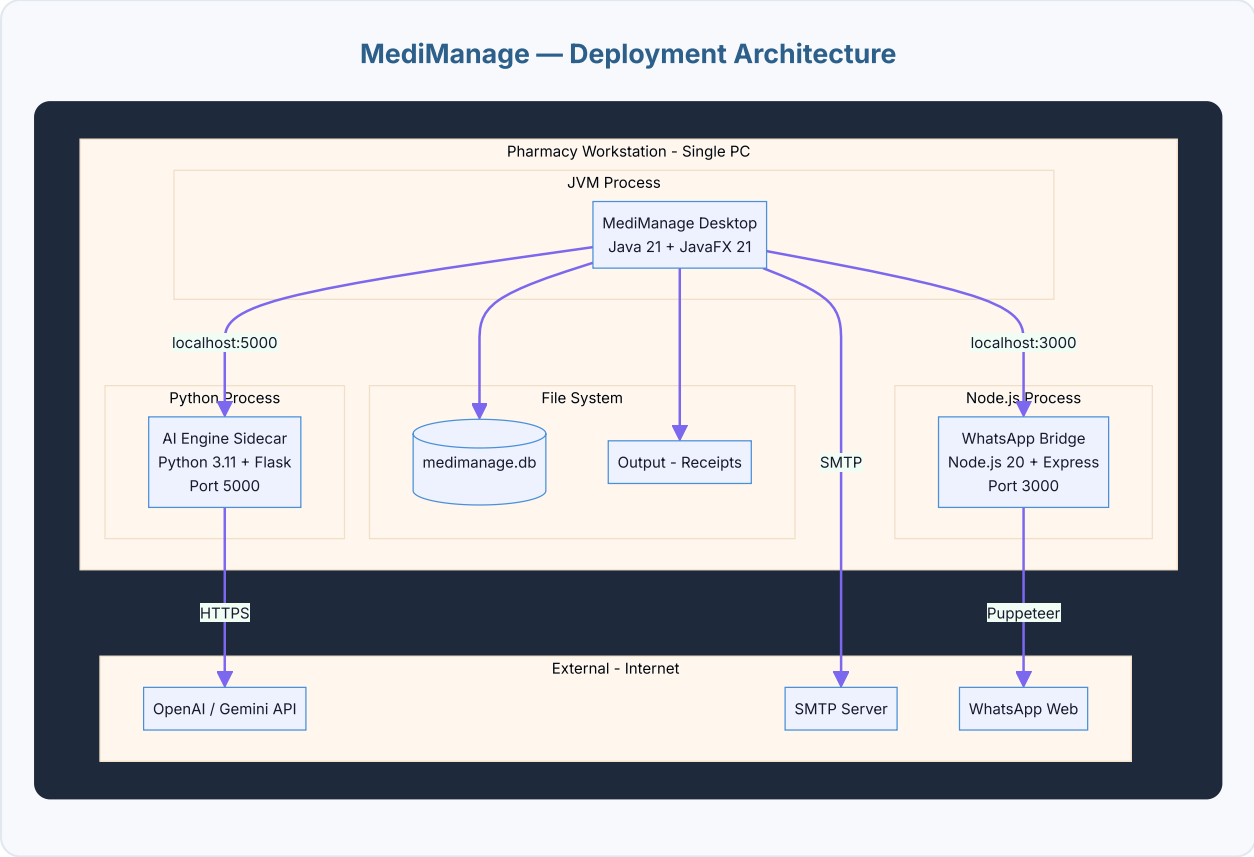
9.1 High-Level Architecture Diagram



9.2 Block Diagram — Three-Tier Component View



9.3 Deployment Diagram



10. Why Did You Choose This Domain?

> **Q: Why did you choose the pharmacy management domain?**

1. **Real-World Impact** — Pharmacies are essential healthcare infrastructure. Improving their operations directly impacts patient safety (preventing expired medicine dispensing) and public health accessibility.
2. **Untapped Market** — While large pharmacy chains use enterprise ERP systems, the majority of **8+ lakh standalone pharmacies in India** still rely on manual processes. There's a massive gap for affordable, easy-to-use digital solutions.
3. **Rich Problem Domain** — Pharmacy management involves complex interconnected operations — inventory with expiry tracking, regulatory compliance (GST), customer health records, supplier logistics, financial tracking — providing excellent learning opportunities across multiple CS domains.
4. **Local-First Architecture** — The pharmacy use case naturally demands offline-first design because many pharmacies are in areas with unreliable internet. This gave me the opportunity to explore modern local-first software architecture.
5. **AI Integration Potential** — Pharmacy data (sales trends, inventory patterns, customer behavior) is ideal for AI-powered insights, allowing integration of modern AI capabilities in a

practical way.

6. **Personal Connection** — Having observed the operational challenges at local pharmacy stores, I identified first-hand the pain points that motivated the development of this solution.

11. How Many Feature Sets?

> **Q: How many feature sets does your project have?**

MediManage has **12 major active feature sets** + 2 planned, organized across 3 tiers:

Core Feature Sets (Offline-First) — 9 sets

#	Feature Set	Key Capabilities
1	**Billing & POS**	Cart, barcode scan, split payments, held orders, PDF receipt/invoice, prescription directions
2	**Inventory**	Medicine catalog, batch tracking (FEFO), stock adjustment, low-stock & expiry alerts
3	**Customer Management**	Profiles, credit ledger, disease/insurance records, loyalty points
4	**Purchase Orders**	Supplier management, PO lifecycle, batch creation on receipt
5	**Reporting & Analytics**	Sales, profit, inventory valuation, GST reports, PDF/Excel/CSV export
6	**Dashboard**	Real-time KPIs, sales trends, top products, alert summary
7	**User & Access Control**	Login, RBAC (Admin/Supervisor/Staff), supervisor approvals
8	**Attendance & Expenses**	Employee attendance, expense categorization
9	**Audit & Security**	Audit logging, BCrypt, input validation

AI-Enhanced (Requires Sidecar) — 1 set

#	Feature Set	Key Capabilities
10	**AI Assistant**	Chat, pharmacy data querying, inventory analysis, cloud AI orchestration

Communication (Requires Internet) — 2 sets

#	Feature Set	Key Capabilities
11	**WhatsApp Bridge**	Invoice delivery, credit follow-ups, template messaging
12	**Email Service**	Invoice emailing, report dispatch scheduling

Planned / Feature-Flagged — 2 sets

#	Feature Set	Status
13	**Subscription Plans**	Schema ready, feature-flagged OFF
14	**MCP Integration**	Server implemented, awaiting client adoption

12. What Is Your Data Size?

> **Q: What is your data size?**

Metric	Value
Database file size	~33 MB (SQLite)
Number of tables	22 tables
Number of views	2 materialized views
Number of indexes	45+ indexes
Seed data — Medicines	1,000+
Seed data — Customers	500+
Seed data — Bills	5,000+
Seed data — Inventory Batches	3,000+
Seed data — Suppliers	50+
Java source files	~70 files
FXML views	18 files
JasperReport templates	3 (invoice, receipt, prescription)
Python AI engine files	~10 files
Node.js bridge files	4 files
Project size (source only)	~5 MB
Project size (with deps)	~350 MB

13. How Did You Collect Data?

> **Q: How did you download or collect data for your project?**

MediManage uses **synthetically generated pharmacy data** rather than downloaded datasets:

1. **Schema-First Design** — Designed the complete database schema based on real-world pharmacy operations research (studying existing billing software, visiting local pharmacies,

analysing GST-compliant invoice requirements).

2. **Seed Database Generation** — Built a dedicated utility class (`GenerateCleanDatabase.java`) that programmatically generates realistic pharmacy data — medicine catalog from common Indian pharmaceutical products, synthetic customer profiles, simulated billing transactions, inventory batches with realistic expiry dates.
3. **Database Initialization** — The `DatabaseUtil.java` utility handles schema creation with constraints, index creation, view creation, and default admin user bootstrap.
4. **Packaged Starter Database** — For installed deployments, a pre-populated `base_medimanage.db` (~33 MB) is bundled so new installations start with a functional demo dataset.
5. **Operational Data Entry** — Since this is a management application (not a data analysis project), the data is operational data that would naturally be entered by pharmacy staff during daily operations. The seed data simulates this usage.

14. Additional Possible Questions

Q: What design pattern did you use?

MediManage follows the **MVC (Model-View-Controller)** architectural pattern:

- **Model**: 19 POJO classes (Medicine, Customer, Bill, etc.)
- **View**: 18 FXML files with CSS styling
- **Controller**: 19 Controller classes

Additional patterns used:

Pattern	Where
DAO Pattern	15 DAO classes for data access abstraction
Service Layer	16+ services for business logic encapsulation
Sidecar Pattern	AI Engine & WhatsApp Bridge isolation
Factory Pattern	StorageFactory for backend selection
Strategy Pattern	ConfigSource for configuration resolution
Observer Pattern	JavaFX property bindings

Q: What is the difference between your system and existing pharmacy software?

Feature	MediManage	Traditional Software
Internet Dependency	Offline-first	Often cloud-dependent
AI Insights	Built-in AI chat	No AI features
Cost	Free / Open-source	₹15,000–₹50,000/year
WhatsApp Integration	Yes (automated)	Manual or absent
Batch Tracking	FEFO-based	Basic or none
Deployment	Single installer	Complex setup

Q: What testing methodology did you follow?

- **Unit Tests** (JUnit 5) — DAO layer, Service layer, Configuration, Feature flags
- **Integration Tests** — Database operations with in-memory SQLite
- **Security Tests** (npm test) — WhatsApp bridge token validation and rate limiting
- **AI Engine Tests** (pytest) — Python sidecar route tests and safety guard tests
- **Regression Tests** — Specific regression suites for critical bugs

Q: How do you handle security?

1. **Password Storage**: BCrypt hashing (never plain text)
2. **Authentication**: Login-based access with session management
3. **Authorization**: Role-Based Access Control (Admin/Supervisor/Staff)
4. **AI Safety**: Input safety guard filters malicious prompts
5. **Sidecar Security**: Localhost-only binding + local admin tokens
6. **Audit Trail**: All critical operations logged in `audit_events` table
7. **Supervisor Approvals**: Sensitive operations require supervisor override

Q: What is MCP (Model Context Protocol)?

MCP is an open protocol that allows AI models to access external tools and data sources. In MediManage, the MCP server exposes pharmacy inventory, customer, sales, and database tools that compatible AI hosts (like Claude Desktop or Cursor) can use to answer pharmacy-related queries with real data context.

Q: Explain the sidecar architecture.

A sidecar is a helper process that runs alongside the main application on localhost:

- The **main application** (JavaFX) handles all UI and core business logic
- The **AI Engine** (Python Flask on port 5000) handles AI orchestration — separated because Python has a richer AI ecosystem
- The **WhatsApp Bridge** (Node.js on port 3000) handles browser automation — separated because Puppeteer requires Node.js
- Sidecars are **started and managed** by the main application with health checks and ownership metadata
- If a sidecar is unavailable, the main app **continues to function** normally

15. 🃏 Quick-Read Cards

Use these as flash cards for last-minute revision before the viva.

🃏 PROJECT IDENTITY

01

Project	MediManage
Type	Local-First Pharmacy Operations Desktop App
Version	7.0.0
Languages	Java 21, Python 3.11, JavaScript (Node.js 20)
Database	SQLite (embedded, ~33 MB)
UI Framework	JavaFX 21 + AtlantaFX theme
Architecture	MVC + Sidecar Pattern

🃏 PROBLEM → SOLUTION

02

Problem	Solution
Manual billing errors	Automated POS with barcode scanning
Expired medicine losses	Batch-level FEFO tracking + expiry alerts
No stock visibility	Real-time inventory dashboard + reorder alerts
Credit leakage	Digital customer ledger with balance tracking
No business insights	AI-powered analytics and trend analysis
Internet dependency	Local-first design with SQLite
Expensive tools	100% open-source, free to deploy

🃏 TECH STACK MAP

03

```
JAVA 21 ──→ Desktop Shell (JavaFX + FXML + CSS)
   ├── SQLite        ──→ Embedded Database (22 tables)
   ├── JasperReports ──→ PDF invoices & receipts
   ├── BCrypt        ──→ Password hashing
   ├── Gson          ──→ JSON serialization
   └── ZXing         ──→ Barcode generation

PYTHON 3.11 ──→ AI Engine Sidecar (Flask on :5000)
   ├── Cloud AI APIs ──→ OpenAI / Gemini
   └── MCP Server    ──→ Model Context Protocol

NODE.js 20 ──→ WhatsApp Bridge (Express on :3000)
   └── whatsapp-web.js ──→ Browser automation
```

🃏 NUMBERS TO REMEMBER

04

Metric	Count
Java Controllers	19
Model Classes	19
DAO Classes	15
Service Classes	16+
FXML Views	18
Database Tables	22
Database Views	2
Database Indexes	45+
Feature Sets	12 active + 2 planned
DB Size	~33 MB
AI API Endpoints	7

🃏 DESIGN PATTERNS USED

05

Pattern	Where
MVC	JavaFX (Model → View/FXML → Controller)
DAO	15 DAO classes abstracting SQLite
Service Layer	Business logic between Controller & DAO
Sidecar	AI Engine & WhatsApp Bridge
Factory	StorageFactory for backend selection
Strategy	ConfigSource for config resolution
Observer	JavaFX property bindings

🃏 KEY DEPENDENCIES

06

Library	Purpose
javafx-* (21.0.2)	Desktop UI framework
sqlite-jdbc (3.46.0)	Database driver
atlantafx-base (2.0.1)	Modern UI theme
jasperreports (6.21.0)	PDF report generation
openpdf (1.3.40)	PDF engine
poi-ooxml (5.2.5)	Excel export
zxing (3.5.3)	Barcode generation
flexmark-all (0.64.8)	Markdown rendering
jbcrypt (0.4)	Password hashing
twilio (10.1.0)	WhatsApp API

🃏 BILLING FLOW — ONE LINER

07

```
Login → Search Medicine (barcode/name) → Add to Cart
→ Select Customer → Choose Payment Mode
→ Checkout → Deduct Stock → Generate Receipt PDF
→ Print / Email / WhatsApp
```

🃏 PURCHASE ORDER FLOW

08

```
Create PO (DRAFT) → Add Items → Submit (ORDERED)
→ Receive Delivery → Update Quantities (PARTIAL/RECEIVED)
→ Auto-Create Inventory Batches → Update Stock
→ PO Complete
```

🃏 SECURITY LAYERS

09

```
Layer 1: BCrypt Password Hashing     (no plaintext)
Layer 2: RBAC Policies               (Admin > Supervisor > Staff)
Layer 3: Supervisor Approval Workflow (sensitive ops need override)
Layer 4: AI Input Safety Guard       (filters malicious prompts)
Layer 5: Localhost-Only Sidecars     (no external exposure)
Layer 6: Local Admin Tokens          (sidecar authentication)
Layer 7: Audit Event Logging         (complete operation trail)
```

🃏 FUTURE WORK — TOP 5

10

Priority	Enhancement
1	Multi-branch cloud sync
2	Subscription & loyalty module activation
3	Mobile companion app
4	Drug interaction checker
5	Predictive demand forecasting with ML

🃏 WHY THIS DOMAIN? — 3 Points

11

1. **Real need:** 8+ lakh Indian pharmacies still use manual registers
2. **Rich complexity:** Combines inventory, billing, CRM, AI, and communications
3. **Offline-first challenge:** Explores modern local-first architecture patterns

🃏 CONCLUSION — 3 Points

12

1. Proved that **local-first + AI** is feasible for pharmacy operations
2. Reduces manual effort by **60–70%** with automated billing & inventory
3. Offers **enterprise-grade features** at zero licensing cost using open-source tools

💡 EXAM TIP

Before the viva: Read Cards 1–3 for a quick refresh. Practice explaining the Architecture Diagram (Section 9.1) on a whiteboard. Be ready to trace through the Billing Sequence Diagram (Section 8.3) step by step.

Document prepared for MediManage internship viva voice examination • Generated April 2026